

4

Graphical Models

Welcome to *Chapter 4*!

So far, we have used graphs mainly to visualize our models. In this chapter, we'll see that from the causal point of view, graphs are much more than just a visualization tool. We'll start with a general refresher on graphs and basic graph theory. Next, we'll discuss the idea of **graphical models** and the role of **directed acyclic graphs (DAGs)** in causality. Finally, we'll look at how to talk about causality beyond DAGs. By the end of this chapter, you should have a solid understanding of what graphs are and how they relate to causal inference and discovery.

This knowledge will give us the foundations to understand *Chapter 5*. This and the next chapter are critical to understanding the very essence of causal inference as understood in this book.

In this chapter, we'll cover the following:

- A refresher on graphs
- Graphical models in causality
- Causal directed acyclic graphs (DAGs)
- Causality beyond DAGs

Graphs, graphs, graphs

This section will be a quick refresher on graphs and basic graph theory. If you're not familiar with graphs – don't worry – you can treat this section as a crash course on the topic.

Let's start!

Graphs can be defined in multiple ways. You can think of them as discrete mathematical structures, abstract representations of real-world entities and relations between them, or computational data structures. What all of these perspectives have in common are the basic building blocks of graphs: **nodes** (also called vertices) and **edges** (links) that connect the nodes.

Types of graphs

We can divide graphs into types based on several attributes. Let's discuss the ones that are the most relevant from the causal point of view.

Undirected versus directed

Directed graphs are graphs with directed edges, while **undirected graphs** have undirected edges. Figure 4.1 presents an example of a directed and undirected graph:

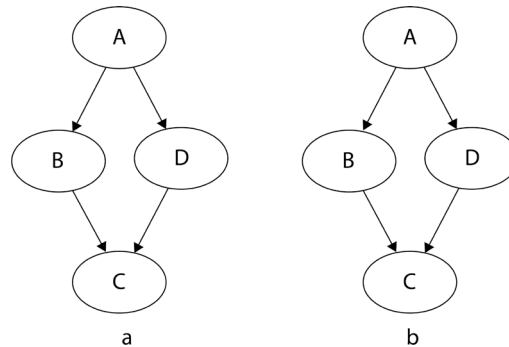


Figure 4.1 – Directed (a) and undirected (b) graphs

As we saw in the previous chapter, we mark the edge direction with an arrow. Lines without arrows denote undirected edges. In the literature, we sometimes also see a line with two arrows (in both directions) to denote an undirected edge or to denote correlation rather than causation (for example, in *structural equation models*).

In certain cases, we might not have full knowledge of the orientation of all the edges in a graph of interest.

When we know all the edges in the graph, but we are unsure about the direction of some of them, we can use **complete partially directed acyclic graphs (CPDAGs)** to represent such cases.

CPDAGs are a special case of a broader class of *partially directed* graphs.

We'll see in *Part 3, Causal Discovery*, that some causal discovery methods can in certain cases only recover partial causal structures from the data. Such structures can be encoded as *partially directed* graphs.

In general, you can also see graphs with different types of edges, denoting different types of relations between nodes. This is often the case in knowledge graphs, network science, and in some applications of graph neural networks. In this book, we'll focus almost exclusively on graphs with one edge type.

Cyclic versus acyclic

Cyclic graphs are graphs that allow for loops. In general, loops are paths that lead from a given node to itself. Loops can be direct (from a node to itself; so-called **self-loops**) or indirect (going through other nodes).

Figure 4.2 presents an example of an **acyclic graph** and a cyclic one:

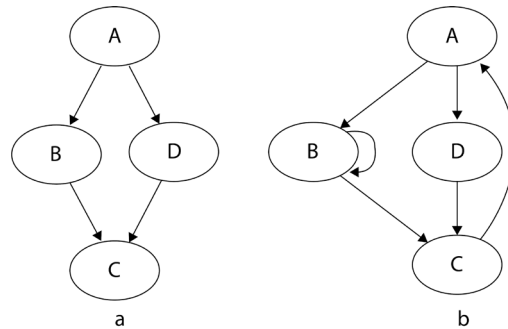


Figure 4.2 – Acyclic (a) and cyclic (b) graphs

The graph on the right side of Figure 4.2 (b) contains two types of loops. There's a self-loop between node B and itself and there are two other larger loops. Can you find them?

Let's start from A. There are two paths we can take. If we move to B, we can either stay at B (via its self-loop) or move to C. From C, we can only get back to A.

If we move from A to D instead, this will also lead us to C and back to A from there.

Most methods that we'll present in this book will assume that the underlying system can be accurately represented by an acyclic graph. That said, there also exist causal methods that support cyclic relationships. We'll discuss them briefly in the last section of this chapter.

Connected versus disconnected

In **connected graphs**, every node has an edge with at least one other node (for example, Figure 4.3 (a)). A **fully-connected graph** contains edges between all possible pairs of variables. **Disconnected graphs** contain no edges:

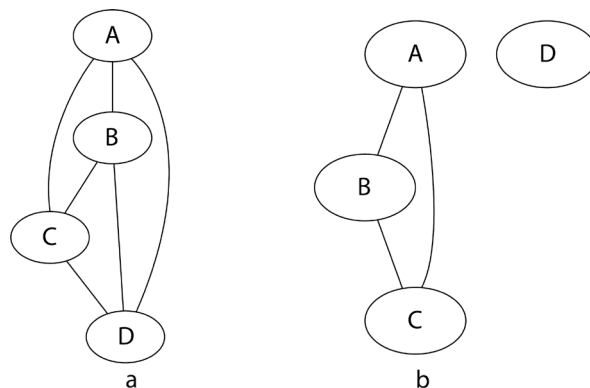


Figure 4.3 – Connected (a) and partially connected (b) graphs

In *Figure 4.3*, we can see a fully-connected graph on the left (*a*) and a partially connected one on the right (*b*). Note that real-world causal graphs are rarely fully connected (that would mean that everything is *directly* causally related to everything else either as a cause or as an effect).

Weighted versus unweighted

Weighted graphs contain additional information on the edges. Each edge is associated with a number (called a **weight**), which may represent the strength of connection between two nodes, distance, or any other metric that is useful in a particular case. In certain cases, weights might be restricted to be positive (for example, when modeling distance). *Unweighted graphs* have no weights on the edges; alternatively, we can see them as a special case of a weighted graph with all edge weights set to 1. In the context of causality, edge weights can encode the strength of the causal effect (note that it will only make sense in the linear case with all the structural equations being linear with no interactions).

Graph representations

Now we're ready to talk about various ways we can represent a graph.

We've seen enough graphs so far to understand how to present them visually. This representation is very intuitive and easy to work with for humans (at least for small graphs), but not very efficient for computers.

People figured out a couple of ways to represent graphs in a computer-readable form. One pretty intuitive way is to represent a graph as a list of nodes and a list of edges. One of the advantages of this method is that it can be easily expanded to add extra information about nodes or edges. It's also relatively human-readable for small graphs. At the same time, it's not very efficient computationally speaking.

To optimize certain types of computations, we can represent a graph as an **adjacency matrix**. This representation preserves the information about the graph structure and – possibly – the strength of connections between the nodes. A limitation of adjacency matrices is that they cannot contain any metadata about nodes or edges, but this can be easily overcome.

Adjacency matrices

An unweighted **adjacency matrix** is a matrix that only contains zeros and ones. One represents an edge, and zero represents a lack of an edge. Nodes are encoded as row and column indices. We usually use zero-indexing, which means that we start counting from zero (we'll refer to the first row of the matrix as the 0th row).

Adjacency matrices are square $M \times M$ matrices where M is the number of nodes. Each positive entry in the matrix encodes an edge between a pair of nodes.

Let's see a couple of examples to make it clearer. We'll start with something very simple:

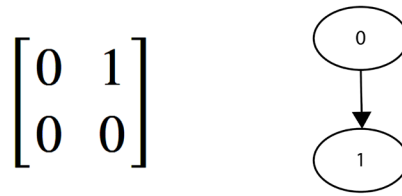


Figure 4.4 – A 2×2 adjacency matrix and the corresponding graph

Figure 4.4 presents a 2×2 adjacency matrix and its corresponding graph. The only non-zero element (1) in the matrix is located in the upper-right corner. Its index is (0, 1) because the element is in the 0th row and the first column. It means that there's only one edge in the graph that goes from node 0 (the row index) to node 1 (the column index). You can verify this in the graph on the right.

Let's see another example:

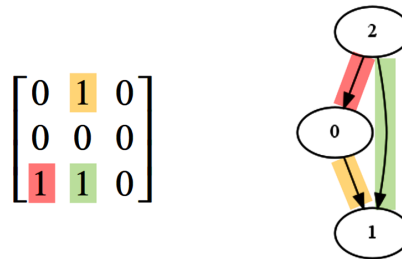


Figure 4.5 – A 3×3 adjacency matrix and the corresponding graph

In Figure 4.5, we see a 3×3 adjacency matrix with three non-zero entries. Their respective indices are (0, 1), (2, 0), and (2, 1). This translates to three edges: from node 0 to node 1, from node 2 to node 0, and from node 2 to node 1.

You can see the respective edges in the corresponding graph. We added color-coding to make the correspondence between the matrix and the graph easier to identify.

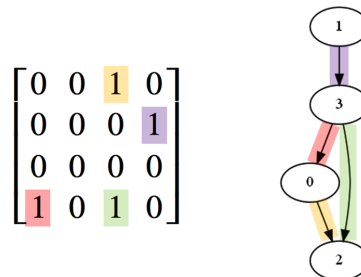


Figure 4.6 – A 4×4 adjacency matrix and the corresponding graph

Figure 4.6 presents a larger 4×4 matrix and the corresponding 4-node graph:

There are four 1-entries that translate to four edges in the graph: from 0 to 2, from 1 to 3, from 3 to 0, and from 3 to 2. The non-zero entries in the matrix and the respective edges are color-coded for your convenience.

Directed and acyclic

Have you noticed that all the matrices in our examples had *zeros on the diagonal*?

This is not by accident. All the preceding matrices represent valid *DAGs* – a type of graph with no cycles and with directed edges only.

How is it related to the values on the diagonal?

Any diagonal entry in a matrix will have an index in the (i, i) form, denoting an edge from node i to itself.

In other words, any 1-entry on the diagonal would denote a self-loop.

This means that a valid *DAG* should always have *zeros on the diagonal*. Self-loops, represented by ones in the diagonal would lead to cyclicity and DAGs are acyclic by definition.

If you're familiar with matrix linear algebra, you might have thought that the fact that a matrix has only zeros in the diagonal provides us with important information about its trace and eigenvalues.

That's great intuition!

This idea is leveraged by some of the **causal discovery** methods to make sure that a graph that we're trying to find is a valid DAG. We'll learn more about these methods in *Part 3, Causal Discovery*.

Graphs in Python

Now we're ready for some practice.

There are many options to define graphs in Python and we're going to practice two of them: using **graph modeling language (GML)** and using adjacency matrices. Figure 4.7 presents a GML definition of a three-node graph with two edges and the resulting graph. The code from Figure 4.7 and the remaining part of this section is available in the notebook for *Chapter 4* in the *Graphs in Python* section (<https://bit.ly/causal-ntbk-04>) :

```
sample_gml = """graph [
directed 1

node [
  id 0
  label "0"
]

node [
  id 1
  label "1"
]

node [
  id 2
  label "2"
]

edge [
  source 0
  target 1
]

edge [
  source 2
  target 1
]

]

"""
```

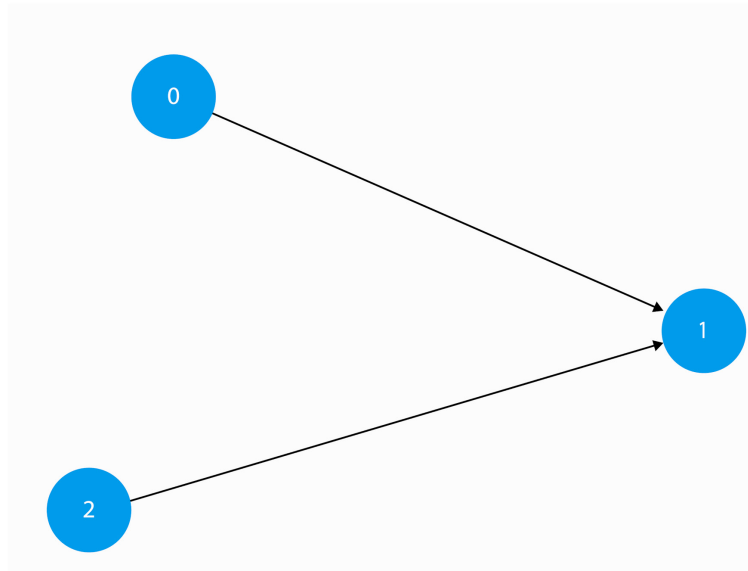


Figure 4.7 – GML definition of a graph and the resulting graph

Graph languages in Python

The GML language is supported by the NetworkX and DoWhy Python libraries. More details on GML and its syntax can be found here: <https://bit.ly/GMLDocs>.

Another popular graph language is DOT. A dedicated Python library called pydot (<https://bit.ly/PyDOTDocs>) allows you to easily read, write, and manipulate DOT graphs in pure Python. DOT is used by `graphviz` and can be used with NetworkX and DoWhy.

GML syntax can be parsed using the NetworkX `parse_gml()` function, which returns a `networkx.classes.digraph.DiGraph` instance (*digraph* is shorthand for *directed graph*). The usage is very simple:

```
graph = nx.parse_gml(sample_gml)
```

GML is pretty flexible, but also pretty verbose. Let's define the same graph using an adjacency matrix:

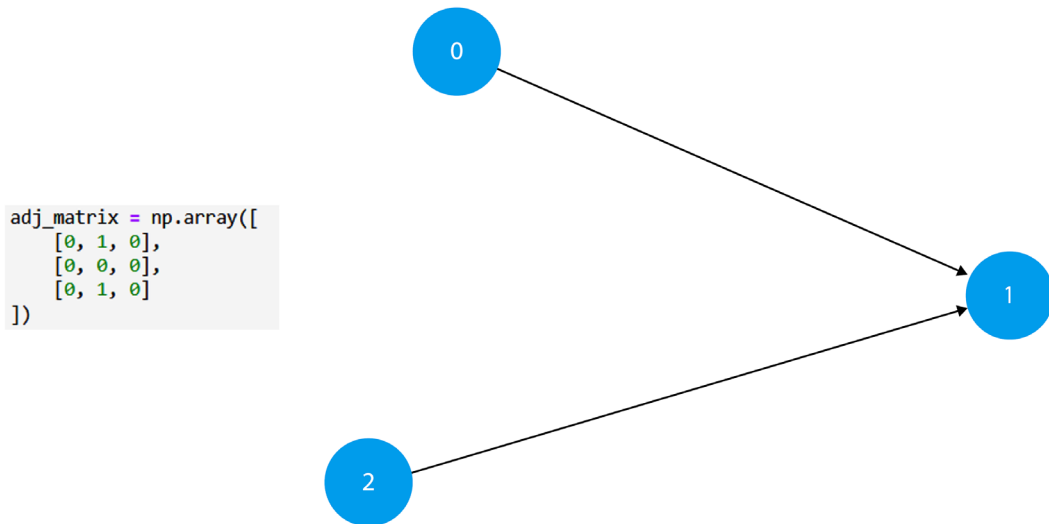


Figure 4.8 – Adjacency matrix of a graph and the resulting graph

Figure 4.8 presents an adjacency matrix created using NumPy and the resulting graph. As you can see, this definition is much more compact. It's also directly usable (without additional parsing) by many algorithms. To build a NetworkX graph from an adjacency matrix, we need just one line of code:

```
graph = nx.DiGraph(adj_matrix)
```

By using `nx.DiGraph`, we tell NetworkX that we want to get a directed graph. To create an undirected one, use the `nx.Graph` object.

To get some practice, create the following graph using GML and an adjacency matrix yourself and visualize it using NetworkX:

- Directed acyclic graph
- Six nodes
- Six edges: (0, 1), (0, 3), (0, 5), (3, 2), (2, 4), (4, 5)

You should get a result similar to the one presented in Figure 4.9:

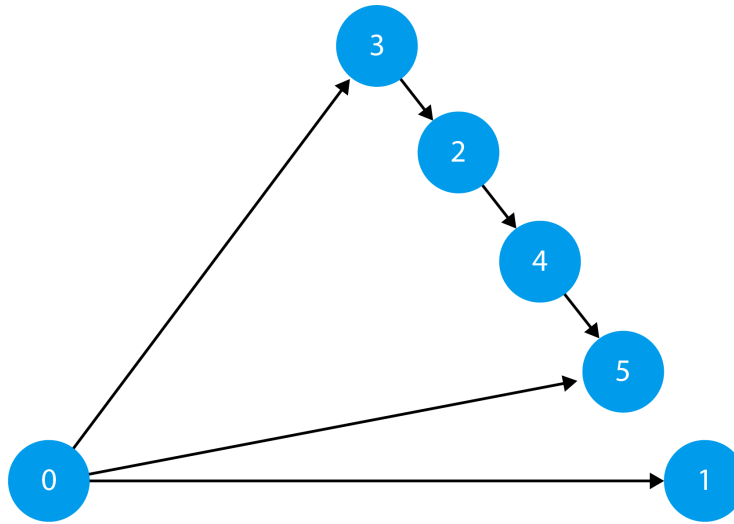


Figure 4.9 – The resulting graph

In this section, we discussed various types of graphs. We learned what a DAG is and how to construct it in Python. We've seen that graphs can be represented in many different ways and used two of these ways (*GML* and *adjacency matrices*) to build our own graphs. We're now ready to discuss graphical models.

What is a graphical model?

In this section, we're going to discuss what **graphical causal models (GCMs)** are and how they can help in causal inference and discovery.

GCMs can be seen as a useful framework that integrates probabilistic, structural, and graphical aspects of causal inference.

Formally speaking, we can define a graphical causal model as a set consisting of a graph and a set of functions that induce a joint distribution over the variables in the model (Peters et al., 2017).

The basic building blocks of GCM graphs are the same as the basic elements of any directed graph: nodes and directed edges. In a GCM, each node is associated with a variable.

Importantly, in GCMs, edges have a strictly causal interpretation, so that $A \rightarrow B$ means that A causes B (this is one of the differentiating factors between causal models and Bayesian networks; Pearl & Mackenzie, 2019, pp. 111-113). GCMs are very powerful because certain combinations of nodes and edges can reveal important information about the data. In other words, the sheer graph structure is in some cases enough to decode information about statistical relationships between variables.

This also works in the other direction – in certain cases, we can infer the true causal graph structure by looking at the statistical relationships alone. The latter is possible when a special assumption called the **faithfulness assumption** and some other conditions are met. Although *faithfulness* seems a relatively light assumption on face value, a deeper examination reveals underlying challenges (Uhler et al., 2013). We’ll learn more about faithfulness in *Chapter 5*.

These properties of GCMs are the basis of many causal inference and discovery methods and can also be leveraged for *domain adaptation* even in the presence of unobserved confounders (Magliacane et al., 2018) – a flexible and very practical extension of the causal discovery toolbox.

To summarize, GCMs can be seen as a useful and powerful framework that unifies probabilistic, structural, and graphical perspectives on causal inference. GCMs are powerful because they offer the possibility to translate between the graphical properties of a model and the statistical properties of the data.

All that said, definitions of GCMs are not entirely consistent in the literature. In most places in this book, we’ll talk about SCMs as consisting of a graph and a set of functional assignments. This will allow us to avoid the confusion related to the inconsistencies in the literature, while preserving the most important elements of the graphical and functional representations.

Now, let’s solidify our knowledge of DAGs.

DAG your pardon? Directed acyclic graphs in the causal wonderland

We’ll start this section by reviewing definitions of causality. Then, we’ll discuss the motivations behind DAGs and their limitations. Finally, we’ll formalize the concept of a DAG.

Definitions of causality

In the first chapter, we discussed a couple of historical definitions of causality. We started with Aristotle, then we briefly covered the ideas proposed by David Hume. We’ve seen that Hume’s definition (as we presented it) was focused on associations. This led us to look into how babies learn about the world using experimentation. We’ve seen how experimentation allows us to go beyond the realm of observations by interacting with the environment. The possibility of interacting with the environment is at the heart of another definition of causality that comes from Judea Pearl.

Pearl proposed something very simple yet powerful. His definition is short, ignores ontological complexities of causality, and is pretty actionable. It goes as follows: *A causes B if B listens to A* (Pearl & Mackenzie, 2019).

What does it mean that one variable *listens* to another? In Pearlian terms, it means that if we change *A*, we also observe a change in *B*.

The concept of a change in one variable leading to a change in another one is inherently related to the logic of interventions. Let's see what happens when we combine this definition with the idea of a DAG.

DAGs and causality

Visualize a simple DAG: $A \rightarrow B \rightarrow C$. Now imagine that we perform an intervention on node B and now the value of B entirely depends on our decision.

In such a case, we expect that C will change its value accordingly, as C listens to B .

On the other hand, under the intervention, A will no longer influence B , because our intervention fully controls the value of B . To reflect this in the graph, we remove the edge from A to B .

The operation of removing the incoming edges from a node that we intervene upon is sometimes referred to as graph mutilation.

Graph mutilation is an intuitive way of presenting the impact of intervention and it reveals a deep insight – that an intervention changes the structure of information flow between variables.

The good news is that we don't always need interventions to alter the information flow in the data. If we know which paths in the graph should remain open and which should be closed, we can leverage the power of **d-separation** and statistical control to close and open the paths in the graph, which will allow us to describe causal quantities in purely statistical terms under certain conditions. Moreover, in some scenarios, it will allow us to learn the structure of the data-generating process from the data generated by this process. We'll learn more about d-separation in *Chapter 6*, and more about learning the structure of the data-generating process in *Part 3, Causal Discovery*.

Before we continue, let's formally define DAGs.

Let's get formal!

DAG G consists of a set of vertices V and a set of directed edges E . Each node V_i is associated with a random variable X_i . Let's denote an edge from a vertex i to another vertex j as $i \rightarrow j$. We call V_i a **parent** of V_j and we call V_j a **child** of V_i . For any directed path $i \rightarrow \dots \rightarrow j$, with $k \in \mathbb{Z}_0^+$ (non-negative) vertices between i and j , we call V_i an **ancestor** of V_j and we call V_j a **descendant** of V_i . Note that because $k \geq 0$, parents are a special case of ancestors and children are a special case of descendants.

By definition, in a DAG, there are no paths that start at vertex i that lead back to vertex i (either directly or indirectly).

Limitations of DAGs

DAGs seem to capture certain intuitions about causality really well. My best guess is that most people would agree that talking about the direction of causal influence makes sense (can you think of *undirected causality*?). At the same time, I believe that fewer people would agree that causal influence cannot be cyclic. Let's see an example.

Imagine an interaction between two partners. One partner says something at time t_0 , the other partner responds at time t_1 . We could build a simple model of a conversation between the partners in the following way:

$$P_{X=1}(t_j) := f(P_{X=2}(t_{j-1}), I_{X=1}(t_{j-1}))$$

$$P_{X=2}(t_{j+1}) := f(P_{X=1}(t_j), I_{X=2}(t_j))$$

In the preceding formula, $P_X(t_j)$ is partner X 's utterance at time t_j , I_X is partner X 's internal state at time t_j , and $f(\cdot)$ is some function (for the sake of this example, we don't care what function exactly).

To be precise, we should also model changes in I_X (internal states of partners), but we'll leave it out for simplicity.

If we run these equations sequentially for a number of steps, we'll see that what partner 1 said at time t_j will become dependent on what they said at time t_{j-2} .

The same is true for partner 2. This constitutes an (indirect) cycle.

Another example – perhaps more intuitive for some readers – comes from the field of economics. When demand for product P grows, the producer might increase the supply in the hope of collecting potential profits from the market. Increased supply might cause a price drop, which in turn can increase demand further.

Theoretically, we could try to model both examples as an *unrolled* sequence of variables, but there are potential problems with this approach. The main one is that we need to produce a new variable for each timestep, creating a very inefficient representation.

Taking a more general perspective, causal DAGs in particular, and SCMs in general, have limitations. As expressed by Peters and colleagues: SCMs are an “abstraction of underlying physical processes – abstraction whose domain of validity as causal models is limited” (Peters et al., 2017). That said, SCMs (but not DAGs) can be adapted to work with cycles (e.g. Mooji & Classen, 2020).

Sources of causal graphs in the real world

We have discussed graphs from several perspectives now, yet we haven't tackled an important practical question: what is the source of causal graphs in the real world?

In this section, we'll provide a brief overview of such sources and we'll leave a more detailed discussion for *Part 3* of the book.

On a high level, we can group the ways of obtaining causal graphs into three classes:

- Causal discovery
- Expert knowledge
- A combination of both

Let's discuss them briefly.

Causal discovery

Causal discovery and **causal structure learning** are umbrella terms for various kinds of methods used to uncover causal structure from observational or interventional data. We devote the entirety of *Part 3* of this book to this topic.

Expert knowledge

Expert knowledge is a term covering various types of knowledge that can help define or disambiguate causal relations between two or more variables. Depending on the context, expert knowledge might refer to knowledge from randomized controlled trials, laws of physics, a broad scope of experiences in a given area, and more.

Combining causal discovery and expert knowledge

Some causal discovery algorithms allow us to easily incorporate expert knowledge as a priority. This means that we can either *freeze* certain edges in the graph or *suggest* the existence or direction of these edges. We will discuss some such approaches in *Part 3* of the book.

Extra – is there causality beyond DAGs?

In this extra section, we'll give a brief overview of some non-DAG-based approaches to causality. This is definitely an incomplete and somehow subjective guide.

Dynamical systems

The scenario with two interacting partners that we discussed in the previous section describes a dynamical system. This particular example is inspired by the research by an American ex-rabbi turned psychologist called John Gottman, who studies human romantic relationships from a dynamical systems point of view (for an overview: Gottman & Notarius, 2000; Gottman et al., 1999).

Dynamical systems are often described using differential equations and cannot be solved analytically (for a toy example of differential equations applied to romantic relationships, check Strogatz, 1988).

Dynamical systems have been extensively studied in physics (Strogatz, 2018), biology (Cosentino & Bates, 2011), and psychology (Nowak & Vallacher, 1998), among other fields.

The dynamical approach is closely related to non-linear dynamics, simulation, chaos theory, and complexity science. Research in these fields is often focused on system-level effects rather than interactions between single variables. An important concept in complexity science is **emergence** – a phenomenon in which we can observe certain properties at the system level that cannot be observed at its constituent parts’ level. This property is sometimes described as a *system being more than the sum of its parts*.

Cyclic SCMs

Because SCMs constitute a very useful framework for causal inference, there are many attempts to generalize it to cyclic cases. For instance, Forré, & Mooij (2017) proposed σ -separation – a generalization of d-separation for cyclical systems. Moreover, the same authors presented a causal discovery algorithm that not only works with cycles, but can also handle latent confounders (Forré, & Mooij, 2018). Interestingly, Mooij & Classen (2020) showed that **FCI** (which stands for **fast causal inference**) – a popular causal discovery algorithm – also gives correct results for data generated with cyclical systems under certain circumstances.

Wrapping it up

We started this chapter by refreshing our knowledge of graphs and learned how to build simple graphs using Python and the NetworkX library. We introduced GCMs and DAGs and discussed some common limitations and challenges that we might face when using them.

Finally, we examined selected approaches to model causal systems with cycles.

Now you have the ability to translate between the visual representation of a graph and an adjacency matrix. The basic DAG toolkit that we’ve discussed in this chapter will allow you to work smoothly with many causal inference and causal discovery tools and will help you represent your own problems as graphs, which can bring a lot of clarity – even in your work with traditional (non-causal) machine learning.

The knowledge you gained in this chapter will be critical to understanding the next chapter and the next two parts of this book. Feel free to review this chapter anytime you need.

In the next chapter, we’ll learn how to use basic graphical structures to understand the fundamental mechanics of causal inference and causal discovery.